CODERHOUSE / DATA ENGINEERING

Redshift Streaming Analytics

Pre-entrega 6 · Nicolás López

Kinesis, Apache Flink, Iceberg, AWS Glue y Amazon Redshift

Implementación preparada. Despliegue y pruebas Redshift pendientes de autorización.

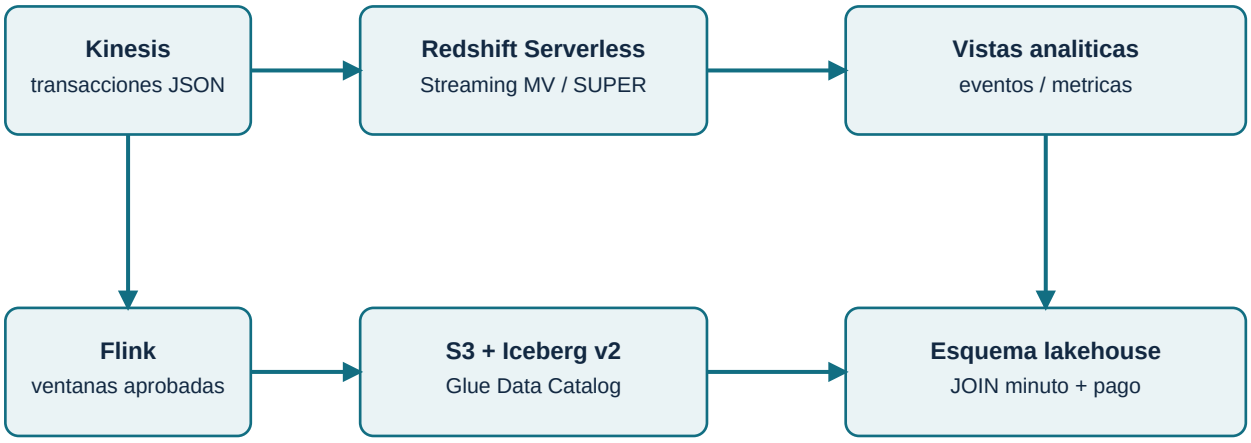
Documento de avance técnico - 6 de noviembre de 2026

Repositorio: https://github.com/nitotito/coderhouse-data-platform

Autor: Nicolás López. Cuenta AWS: 672049777597. Región: us-east-1. Rama: `feature/redshift-streaming-analytics`. **Estado: implementación preparada; despliegue y validación funcional Redshift pendientes de autorización y permisos.**

Objetivo y arquitectura

Consultar eventos recibidos directamente de Kinesis y contrastar sus agregaciones con la tabla Iceberg de la entrega 5, desde una misma base Redshift.



Ramas existentes: Kinesis / Flink / Lakehouse. Redshift: pendiente de despliegue.

Los datos recientes se definen por su llegada a Kinesis; pueden contener eventos con event time histórico. No se unen filas transaccionales con agregados sin agrupar primero. El JOIN es una comparación de dos mediciones del mismo conjunto, no una suma ni una unión de historias solapadas.

Inventario comprobado y compatibilidad

El 06/11/2026 se consultaron AWS STS, Glue, Kinesis, EC2 y S3 con el perfil `coderhouse-dev`: cuenta 672049777597; VPC `vpc-0b85171c643372dba`; subredes `subnet-0e484b46487a0220c` (us-east-1a) y `subnet-001ae24851059e5ed` (us-east-1b), con 251 IP libres cada una. Stream `dev-transactions-stream`: ACTIVE, 2 shards, retención 24 horas, cifrado alias/aws/kinesis. Bucket `coderhouse-data-lake-672049777597`: versionado Enabled. Flink `coderhouse-dev-transactions-flink`: READY. Glue mantiene la tabla ICEBERG y el metadata_location del snapshot probado en PE5.

La primera consulta KMS DescribeKey fue denegada; el error identificó el ARN de la clave `arn:aws:kms:us-east-1:672049777597:key/8cd750a2-438b-42dc-935d-24f3cb6855f4`. Tras habilitar permisos de inventario, DescribeKey confirmó ese ARN y las consultas ListWorkgroups/DescribeClusters devolvieron listas vacías. El ensayo y su retiro fueron autorizados. El apply se detuvo en CreateRole/CreateSecurityGroup por permisos del operador todavía faltantes; no se creó ningún recurso temporal. La política actualizada está en `docs/redshift-test-operator-access.json`.

Las pruebas reales PE5 están en el índice anterior. Son antecedentes, no resultados Redshift. La tabla tiene seis columnas y agregados aprobados por `window_start`, `window_end`, `payment_method`, con `transaction_count`, `total_amount decimal(38,2)` y `average_amount decimal(38,6)`. Iceberg format-version 2 y Parquet son compatibles con lectura Redshift. Serverless usa su propio cómputo para leer el lake; no se agrega el cargo Spectrum por TB de los clusters provisionados RA3. No se crean estadísticas Glue de pago.

Decisión de infraestructura y costos

Se elige Serverless de 4 RPU para un ensayo pequeño, sin reservas ni clúster permanentemente encendido. Un provisionado permite pausa/reanudación, pero agrega administración de nodos y capacidad menos flexible. Serverless **no tiene la operación StopApplication de Flink**: deja de consumir cómputo al quedar inactivo, pero el almacenamiento y PrivateLink siguen facturándose.

AWS admite dos AZ sin EVR desde julio de 2025. Se reutilizan las subredes existentes sin modificar el módulo network. No hay NAT ni Internet Gateway en esas rutas: se prepara un endpoint de interfaz Kinesis en ambas AZ, DNS privado, HTTPS y SG de origen Redshift. Data API permite ejecutar SQL sin abrir TCP 5439 ni publicar el workgroup. S3 gateway endpoint existente permanece igual; sin EVR las lecturas del lake son administradas por Redshift. No se agregan endpoints Glue/KMS de pago.

Estimación orientativa on-demand us-east-1, consultada en páginas oficiales:

Concepto	Configuración	Estimación USD sin impuestos
Redshift activo	Base y máximo 4 RPU, 0.375 USD/RPU-h	1.50 por hora activa; 0.75 por 30 min
PrivateLink Kinesis	Dos AZ, 0.01 USD por AZ-h	0.02/h mientras exista, más tráfico
Tráfico PrivateLink	Primer PB, 0.01 USD/GB	Despreciable para el lote pequeño; no cero garantizado
Límite diario	8 RPU-h, acción deactivate	Aproximadamente 3 USD de cómputo; no un tope monetario absoluto
Almacenamiento	MV, metadatos y backups retenidos	Variable; medir, no se asume gratuito

Se proyecta aproximadamente **0.77-1.52 USD para 30-60 minutos activos**, más almacenamiento y tráfico, sin créditos ni impuestos. El endpoint cobra horas parciales completas. Si permanece 730 horas: aproximadamente 14.60 USD/mes aun sin consultas. El límite diario se reinicia y no cubre storage/endpoints ni garantiza detener el gasto exactamente al centavo. Los cargos Kinesis/S3 previos continúan; Flink no se inicia en la prueba propuesta.

Terraform y despliegue

`infra/redshift.tf` habilita `infra/modules/redshift/` mediante `enable_redshift=false` por defecto. Los ocho recursos nuevos son rol IAM, política del rol, dos security groups, endpoint Kinesis, namespace, workgroup y usage limit. El plan realizado propone **8 altas, 0 cambios, 0 bajas**. El primer apply autorizado falló antes de crear recursos por permisos IAM/EC2. El backend y todas las direcciones de entregas previas se conservan.

```
terraform -chdir=infra init
terraform -chdir=infra fmt -check -recursive
terraform -chdir=infra validate
terraform -chdir=infra plan -var='enable_ingestion=true' -var='enable_redshift=true' `
```

```
  -var='redshift_kinesis_kms_key_arn=arn:aws:kms:us-east-1:672049777597:key/8cd750a2-438b-42dc-
935d-24f3cb6855f4' `
  -out=checkpoint6.tfplan
# Solo después de revisar inventario, permisos, costo y recibir autorización:
terraform -chdir=infra apply checkpoint6.tfplan
terraform -chdir=infra output -json redshift
```

Antes del apply, confirmar ausencia o reutilización/importación de un workgroup equivalente. No importar automáticamente otro warehouse sin revisar su dueño. Tras el apply aprobado, conservar `enable_ingestion=true`, `enable_redshift=true` y el ARN KMS en un tfvars local ignorado. Un plan futuro con el valor default false propondría eliminar el módulo; no ejecutarlo sin aprobación específica.

IAM, autenticación y rol SQL

El rol AWS `coderhouse-dev-analytics-execution` se confía a ambos principales Redshift, restringidos por cuenta y ARN de workgroups de la región. AWS asigna UUID al workgroup, por eso el ARN de confianza usa `workgroup/*` sin dependencia circular. La política permite lectura de un stream, decrypt de una clave con contexto de ese stream, Glue Get de la base/tabla y S3 solo en el prefijo Iceberg. ListStreams requiere Resource '*'; GetTables admite descubrir nombres del namespace, pero GetTable y S3 permanecen limitados a la tabla. GetDatabases no es necesario al indicar explícitamente la base y no se concede para enumerar otros catálogos. Si Lake Formation gobierna la ubicación, revisar grants adicionales antes de usarlos; no se agregan permisos globales preventivos.

El operador usa su perfil AWS y Redshift Data API por HTTPS, credenciales IAM temporales y GetCredentials. No hay passwords en Terraform, SQL o scripts. El principal de despliegue necesita privilegios SQL para crear esquemas, MV y roles; GetCredentials por sí solo no acredita esos privilegios. Verificar `current_user` y realizar bootstrap con el administrador de namespace si hace falta.

`checkpoint6_analyst` es un **rol de base de datos**, distinto del IAM execution role. Solo obtiene USAGE en analytics y SELECT en tres vistas publicadas. No obtiene REFRESH ni CREATE ni acceso al esquema raw. La asignación a un usuario autenticado real se deja explícita y no se inventa un usuario del alumno. Validar privilegios positivos/negativos con esa identidad antes de dar por completada la seguridad funcional.

SQL, JSON y refresco

Los archivos 01 a 07 separan esquemas, MV, métricas, Iceberg, seguridad, validación y monitoreo. checkpoint_6_complete.sql se genera de esos originales, en orden. Ejecutar una vez como propietario: CREATE MV y CREATE ROLE no se ocultan tras DROP para permitir re-ejecuciones.

La MV raw conserva `kinesis_data`, shard/sequence, arrival/refresh time, `valid_json` y `JSON_PARSE` a SUPER protegido por CASE/CAN_JSON_PARSE. La vista posterior extrae cinco campos reales, convierte a importes decimales y timestamp UTC, y clasifica errores. TRY_CAST recibe NULLIF para cadenas vacías; los importes canónicos aceptan dígitos y hasta dos decimales, sin redondear silenciosamente valores de mayor escala. Es un contrato explícito más estricto para formatos como notación científica. Nuevas propiedades JSON no alteran el esquema publicado. Se aceptan timestamps ISO con zona y epoch de 13 dígitos en milisegundos.

La MV de streaming no hace JOIN. Las métricas se calculan en una vista SQL normal, evitando prometer mantenimiento incremental para conversiones desde SUPER. Cuenta aprobadas/rechazadas, total y promedio por minuto/método. El JOIN agrupa ambos lados al mismo nivel antes de unir, compara solo aprobadas y muestra diferencias. No suma un total histórico una vez por transacción.

AUTO REFRESH NO limita actividad de laboratorio: se ejecuta REFRESH bajo demanda. Una ingesta nueva puede requerir más de un refresco; el smoke espera hasta 180 s y deja 15 s entre intentos, no un scheduler de un segundo. La latencia automática no se ha medido ni se declara SLA. Si se aprueba una demo continua, habilitar AUTO REFRESH YES, medir con la carga real y volver a NO al terminar. La retención Kinesis de 24 h limita recuperación de pendientes; recrear la MV pierde su checkpoint y puede reingerir registros retenidos. No hay deduplicación de negocio por transaction_id ni garantía exactly-once entre los dos pipelines.

Prueba reproducible y monitoreo

```
python scripts/redshift-checkpoint6.py              # genera SQL, sin llamadas AWS
# Solo con despliegue/costos autorizados y permisos SQL confirmados:
python scripts/redshift-checkpoint6.py --execute --setup
python scripts/redshift-checkpoint6.py --execute --smoke
```

La prueba usa una MV desplegada, envía seis eventos válidos (tres actuales y tres con event time del snapshot PE5), dos JSON con campos inválidos y un JSON malformado. Verifica ocho filas JSON, dos errores de campos, un JSON inválido conservado, métricas actuales: count 3, approved 2, rejected 1, total 1029.30, approved_amount 30.30. El JOIN histórico debe producir una fila con dos aprobadas, 30.30 a ambos lados y diferencias cero. Son **resultados esperados**, no observados. El marcador rechazado de la prueba PE5 no forma parte de este lote. Flink debe estar READY, se verifica antes de enviar. No se reescribe el lake ni se inicia Flink.

El runner guarda SQL ejecutado, estado y resultados paginados de Data API con timeouts. `observation_upper_bound_seconds` incluye envío, refresco y consultas: no es latencia aislada del motor. Si falla, guarda last-failure.json y lanza error. Un PASSED prueba datos/métricas/JOIN, no sustituye la prueba de permisos del analista ni la revisión visual. Las vistas del motor deben ejecutarse realmente antes de declarar compatibilidad funcional final.

Antes de publicar el lote, el runner exige que no haya registros de la dimensión histórica en la MV: evita repetir la muestra y duplicar sus métricas. Si hubo un ensayo anterior o parcial, inspeccionar sus evidencias y elegir otra muestra histórica; no borrar/recrear automáticamente la MV ni reenviar a ciegas.

SYS_STREAM_SCAN_STATES muestra progreso por shard; SYS_STREAM_SCAN_ERRORS contiene fallos del escaneo, no todos los errores de negocio. Estos se cuentan en `validation_error`. SYS_MV_REFRESH_HISTORY permite ver Manual/Auto, estado, inicio, final y duración en microsegundos. SYS_MV_STATE registra eventos de cambio de estado, no un heartbeat. La edad arrival→consulta incluye espera por refresco; refresh_time es el comienzo y no garantiza visibilidad de cada registro. Usuarios regulares ven sus propios registros de sistema; un conjunto vacío no demuestra ausencia universal de errores.

Evidencias y estado real

Índice checkpoint 6. Hay inventario API y plan de Terraform; están pendientes deployment, ejecución SQL, identidad analista, consulta histórica/JOIN, latencia observada y capturas Redshift. La captura Glue de PE5 puede reutilizarse identificando su fecha, sin presentarla como nueva.

El PDF se genera con estas limitaciones explícitas. No está presentado como validación funcional final.

El análisis local SQLGlot con dialecto Redshift interpretó 28 de 32 sentencias; dos CREATE EXTERNAL SCHEMA, CREATE ROLE y REFRESH MATERIALIZED VIEW quedan fuera de su cobertura. No hubo errores de parseo en las otras 28, lo que no verifica tipos, permisos ni objetos reales. Pasaron tres tests locales del runner: paginación preservando decimales y NULL, propagación de fallos del motor y cancelación por timeout. No llaman AWS ni reemplazan el ensayo funcional.

Cierre de costos sin afectar entregas previas

Tras el ensayo: dejar AUTO REFRESH NO, cancelar consultas activas, cerrar sesiones y comprobar SYS_SERVERLESS_USAGE/consola para observar inactividad. No existe pause/stop de Serverless equivalente al provisionado. Inactividad no elimina el cargo de almacenamiento ni el endpoint privado.

Proponer al propietario retirar únicamente `module.redshift[0]`: generar plan con enable_redshift=false y enable_ingestion=true, revisar que TODAS las bajas pertenezcan al módulo Redshift y solicitar aprobación de ese plan antes del apply. Decidir backup/snapshot/export antes de eliminar el namespace: puede perderse el historial local de la MV. No ejecutar terraform destroy, no eliminar Kinesis, Flink, Glue, Iceberg, VPC ni el bucket compartido. El endpoint de interfaz propio debe incluirse en el retiro para evitar su cargo permanente. No se ejecutó retiro.

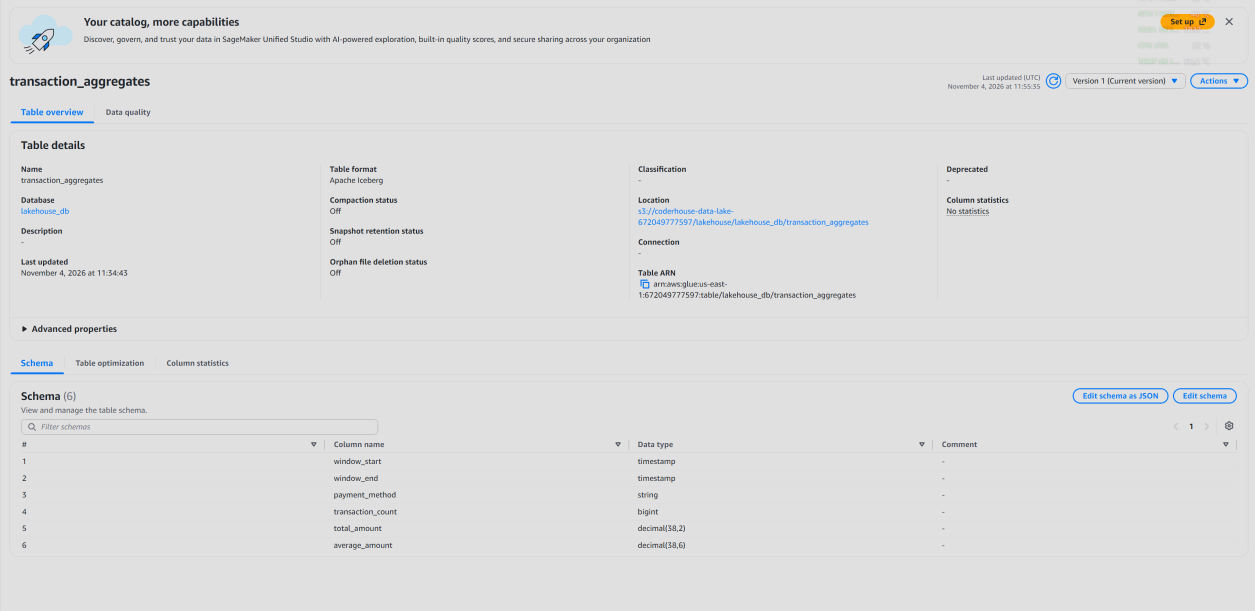
Referencias oficiales

- Ingesta desde Kinesis y metadatos
- Restricciones de Streaming Ingestion
- Iceberg con Redshift
- Capacidad Serverless y 4 RPU
- Serverless con dos AZ sin EVR
- Precios Redshift
- Precios PrivateLink
- Facturación on-demand y límites
- Trust IAM Serverless
- Prevención de confused deputy
- TRY_CAST
- SYS_STREAM_SCAN_STATES
- SYS_STREAM_SCAN_ERRORS
- SYS_MV_REFRESH_HISTORY

- SYS_MV_STATE
- Repositorio

Evidencia real del Lakehouse existente

Captura proporcionada por el propietario en la entrega 5. Confirma catálogo, formato Iceberg y esquema; no demuestra un despliegue de Redshift.



Antecedente PE5: Athena verificó dos aprobadas, total 30.30 y promedio 15.150000. Snapshot 8171898520026477263. Nuevos resultados Redshift: pendientes.

Resultados locales y pendientes

Terraform fmt y validate: correctos. Plan: ocho altas, cero cambios, cero bajas. Parser SQL local: cobertura parcial; las extensiones específicas de AWS requieren ejecución real. El runner compila y su modo predeterminado no llama AWS.

Conclusiones técnicas

La separación de ingesta, validación y métricas reduce el riesgo de bloquear el stream por conversiones. La conciliación por minuto y método respeta la granularidad de Iceberg. El diseño conserva las entregas previas. Falta desplegar, confirmar privilegios SQL, ejecutar el ensayo y adjuntar capturas Redshift para aceptar funcionalmente la entrega.

Anexo: SQL reproducible

01_schemas.sql

```
-- Execute as deployment owner, database analytics, region us-east-1.
-- No CREATE EXTERNAL DATABASE: Glue already exists and remains Terraform-owned.
SET timezone TO 'UTC';
CREATE SCHEMA IF NOT EXISTS streaming;
CREATE SCHEMA IF NOT EXISTS analytics;
CREATE EXTERNAL SCHEMA IF NOT EXISTS kinesis_source
FROM KINESIS IAM_ROLE default REGION 'us-east-1';
CREATE EXTERNAL SCHEMA IF NOT EXISTS lakehouse
FROM DATA CATALOG DATABASE 'lakehouse_db'
IAM_ROLE default REGION 'us-east-1';
```

02_streaming.sql

```
-- Exactly one base MV consumes the existing stream. Refresh is explicitly manual.
-- Keep invalid JSON bytes for diagnostics; CASE guards JSON_PARSE.
-- Business conversions are downstream, so a bad amount cannot block ingestion.
CREATE MATERIALIZED VIEW streaming.transactions_raw
AUTO REFRESH NO AS
SELECT approximate_arrival_timestamp, refresh_time, partition_key,
       shard_id, sequence_number, kinesis_data,
       CAN_JSON_PARSE(kinesis_data) AS valid_json,
       CASE WHEN CAN_JSON_PARSE(kinesis_data)
            THEN JSON_PARSE(kinesis_data) ELSE NULL END AS payload
FROM kinesis_source."dev-transactions-stream";
```

03_analytics.sql

```
-- SUPER paths correspond to real transaction JSON, not Java POJO field names.
-- Empty strings are normalized before TRY_CAST. Objects/arrays become NULL.
CREATE OR REPLACE VIEW analytics.transactions_typed AS
WITH fields AS (
  SELECT approximate_arrival_timestamp, refresh_time, shard_id, sequence_number,
         valid_json,
         NULLIF(TRIM(payload.transaction_id::varchar), '') AS transaction_id,
         NULLIF(TRIM(payload.payment_method::varchar), '') AS payment_method,
         LOWER(NULLIF(TRIM(payload.status::varchar), '')) AS status,
         NULLIF(TRIM(payload.amount::varchar), '') AS amount_text,
         NULLIF(TRIM(payload."timestamp"::varchar), '') AS timestamp_text
  FROM streaming.transactions_raw
), converted AS (
  SELECT *, TRY_CAST(amount_text AS decimal(20,4)) AS precise_amount,
         CASE WHEN timestamp_text ~ '^[0-9]{13}$'
              THEN DATEADD(millisecond, MOD(TRY_CAST(timestamp_text AS bigint),86400000)::integer,
                   DATEADD(day, (TRY_CAST(timestamp_text AS bigint)/86400000)::integer,
                           TIMESTAMP '1970-01-01 00:00:00'))
              ELSE TRY_CAST(timestamp_text AS timestamptz) AT TIME ZONE 'UTC'
         END AS event_time
  FROM fields
)
SELECT approximate_arrival_timestamp, refresh_time, shard_id, sequence_number,
       transaction_id, payment_method, status, event_time,
       TRY_CAST(precise_amount AS decimal(18,2)) AS amount,
       CASE WHEN NOT valid_json THEN 'invalid_json'
            WHEN transaction_id IS NULL OR payment_method IS NULL OR status IS NULL
              THEN 'missing_required_field'
            WHEN amount_text !~ '^[0-9]+([.][0-9]{1,2})?$'
              OR precise_amount IS NULL OR precise_amount < 0
              OR precise_amount <> ROUND(precise_amount,2)
              OR TRY_CAST(precise_amount AS decimal(18,2)) IS NULL THEN 'invalid_amount'
            WHEN event_time IS NULL THEN 'invalid_timestamp'
            ELSE NULL END AS validation_error
FROM converted;
```

```
-- Ordinary aggregate view avoids assumptions about incremental eligibility of
-- SUPER extraction/timestamp conversions in a nested MV. Always reads base state.
CREATE OR REPLACE VIEW analytics.transactions_per_minute AS
SELECT DATE_TRUNC('minute',event_time) AS window_start, payment_method,
       COUNT(*) AS transaction_count,
       SUM(amount) AS total_amount,
       AVG(amount)::decimal(38,6) AS average_amount,
       SUM(CASE WHEN status='approved' THEN 1 ELSE 0 END) AS approved_count,
       SUM(CASE WHEN status='rejected' THEN 1 ELSE 0 END) AS rejected_count,
       SUM(CASE WHEN status='approved' THEN amount ELSE 0 END) AS approved_amount
FROM analytics.transactions_typed
WHERE validation_error IS NULL
GROUP BY 1,2;
```

04_iceberg.sql

```
-- Iceberg contains approved aggregates, NOT individual transactions.
-- Both sides are collapsed to exactly one row per UTC minute/payment method.
-- Historical duplicates are summed, not hidden: drift should expose replay.
-- Late data and Flink watermark/checkpoint timing can temporarily cause drift.
CREATE OR REPLACE VIEW analytics.recent_vs_iceberg AS
WITH recent AS (
  SELECT DATE_TRUNC('minute',event_time) AS window_start, payment_method,
         COUNT(*) AS approved_count, SUM(amount) AS approved_amount
  FROM analytics.transactions_typed
  WHERE validation_error IS NULL AND status='approved'
  GROUP BY 1,2
), historical AS (
  SELECT window_start, payment_method,
         SUM(transaction_count) AS approved_count, SUM(total_amount) AS approved_amount
  FROM lakehouse.transaction_aggregates
  GROUP BY 1,2
)
SELECT r.window_start, r.payment_method,
       r.approved_count AS recent_approved_count,
       h.approved_count AS iceberg_approved_count,
       r.approved_amount AS recent_approved_amount,
       h.approved_amount AS iceberg_approved_amount,
       r.approved_count-h.approved_count AS count_difference,
       r.approved_amount-h.approved_amount AS amount_difference
FROM recent r JOIN historical h
ON r.window_start=h.window_start AND r.payment_method=h.payment_method
WITH NO SCHEMA BINDING;
```

05_security.sql

```
-- Run once as owner. AWS IAM role grants service access; this DB role grants SQL.
CREATE ROLE checkpoint6_analyst;
REVOKE ALL ON SCHEMA streaming FROM PUBLIC;
REVOKE ALL ON SCHEMA analytics FROM PUBLIC;
REVOKE ALL ON SCHEMA kinesis_source FROM PUBLIC;
REVOKE ALL ON SCHEMA lakehouse FROM PUBLIC;
GRANT USAGE ON SCHEMA analytics TO ROLE checkpoint6_analyst;
GRANT SELECT ON analytics.transactions_typed TO ROLE checkpoint6_analyst;
GRANT SELECT ON analytics.transactions_per_minute TO ROLE checkpoint6_analyst;
GRANT SELECT ON analytics.recent_vs_iceberg TO ROLE checkpoint6_analyst;
-- Analyst cannot refresh MVs, create schemas, read raw bytes, or assume IAM role.
-- Explicitly bind a real authenticated DB identity after approving membership:
-- GRANT ROLE checkpoint6_analyst TO "IAM:analyst-name";
```

06_validation.sql

```
-- Manual sequence: publish test records, refresh once, query; no one-second loop.
REFRESH MATERIALIZED VIEW streaming.transactions_raw;
SELECT transaction_id,event_time,amount,payment_method,status,validation_error
FROM analytics.transactions_typed ORDER BY approximate_arrival_timestamp DESC LIMIT 20;
SELECT * FROM analytics.transactions_per_minute ORDER BY window_start DESC LIMIT 20;
SELECT * FROM lakehouse.transaction_aggregates
```

```
WHERE window_start >= TIMESTAMP '2026-11-04 00:00:00'
  AND window_start < TIMESTAMP '2026-11-05 00:00:00';
-- A comparison, not a UNION of overlapping histories; do not add the two totals.
SELECT * FROM analytics.recent_vs_iceberg ORDER BY window_start DESC LIMIT 20;
SELECT * FROM svv_role_grants WHERE role_name='checkpoint6_analyst';
SELECT * FROM svv_relation_privileges WHERE identity_name='checkpoint6_analyst';
```

07_monitoring.sql

```
-- Serverless-compatible SYS views. Owner sees their own refreshes; superuser all.
SELECT * FROM sys_stream_scan_states
WHERE external_schema_name='kinesis_source' ORDER BY record_time DESC LIMIT 20;
SELECT * FROM sys_stream_scan_errors
WHERE external_schema_name='kinesis_source' ORDER BY record_time DESC LIMIT 20;
SELECT mv_name,refresh_type,status,start_time,end_time,duration
FROM sys_mv_refresh_history
WHERE schema_name='streaming' ORDER BY start_time DESC LIMIT 20;
SELECT * FROM sys_mv_state ORDER BY start_time DESC LIMIT 20;
SELECT COUNT(*) AS stored_records, MAX(approximate_arrival_timestamp) AS last_arrival,
       MAX(refresh_time) AS last_ingest_refresh
FROM streaming.transactions_raw;
SELECT validation_error,COUNT(*) AS records
FROM analytics.transactions_typed GROUP BY 1;
-- refresh_time is refresh start, not record visibility time. Negative values are
-- possible for records arriving during a refresh. Data API observation separately
-- measures upper-bound end-to-end latency; never present this as an SLA.
SELECT shard_id,sequence_number,approximate_arrival_timestamp,refresh_time,
       DATEDIFF(millisecond,approximate_arrival_timestamp,GETDATE()) AS arrival_age_ms
FROM streaming.transactions_raw ORDER BY approximate_arrival_timestamp DESC LIMIT 20;
```